Predictive Modeling: Hyper-parameter Optimization

CIS 635 — Knowledge Discovery & Data Mining

Tuning Decision Trees and Random Forests

Course Reading Material

© Dr. Kamrul Hasan, Grand Valley State University. All rights reserved.
For permissions or questions regarding the use of this material, contact hasaka@gvsu.edu.

Please cite [7] this material when using or reproducing it.

Unit: Predictive Modeling — Hyper-parameter Optimization

This is an introductory reading on hyper-parameter optimization. It builds on **Predictive Modeling: Regression** and **Predictive Modeling: Classification**, whose train/validation/test protocol, cross-validation, leakage rules, and evaluation metrics it reuses rather than repeats. Its practical starting point is Koehrsen's tutorial on tuning random forests [8], extended here to single decision trees.

Contents

1 Hyper-parameter Optimization in the KDD Process **2**

2 Parameters and Hyper-parameters **3**

3 The Validation Protocol **3**

4 Validation Curves **4**

5 Search Strategies **5**

6 Python Implementation **5**

7 Evaluating the Search Honestly **7**

8 Common Pitfalls **7**

9 Key Takeaways **8**

Notation and Terminology Quick Reference

The table below collects the recurring terms of this reading for quick lookup. Each term is developed fully in the section listed. Terms shared with the Regression and Classification readings — training/validation/test sets, cross-validation, baseline, overfitting, stratification, recall, ROC AUC — appear in those readings' quick-reference tables.

Term / symbol	Short definition	§
Parameter	A quantity learned from the data during training (e.g., a tree's split thresholds)	2.1
Hyper-parameter	A setting fixed before training that controls how parameters are learned	2.1
Validation curve	Training and CV score versus one hyper-parameter, others held fixed	4
Grid search	Evaluate every combination in a list of values (`GridSearchCV`)	5.1
Random search	Evaluate a fixed number of sampled combinations (`RandomizedSearchCV`)	5.2
Selection bias	Optimism of the best CV score when many candidates are compared	7.1
Nested CV	An outer CV loop that scores the whole tuning procedure	7.1

Learning Objectives

This reading assumes the Regression and Classification readings. After completing it, you should be able to:

- Distinguish parameters from hyper-parameters, and name the hyper-parameters that matter most for decision trees and random forests.
- Compare hyper-parameter settings with stratified k-fold cross-validation on training data only, and estimate what a search will cost.
- Read a validation curve, and choose between grid search, random search, and coarse-to-fine refinement.
- Report an honest performance estimate of a tuned model, using a single test evaluation or nested cross-validation.

1 Hyper-parameter Optimization in the KDD Process

Every algorithm in the Classification reading has knobs: C for logistic regression, k for kNN, depth for a decision tree. Koehrsen compares these knobs to the dials on a radio that must be adjusted until the signal comes through clearly [8]. **Hyper-parameter optimization (HPO)**, or tuning, is the systematic search for the settings that make a model generalize best to new data. Within the KDD process [6], it is part of the modeling step, after the data are prepared and the features are reasonably settled.

Tuning is not the first lever to pull: more data and better features usually pay off more per hour of effort [8]. In Koehrsen's temperature-forecasting example, more data and feature selection reduced the error by about 17.7%, and tuning then reduced it by a further 6.7% [8]. Tune after the data are in good shape, and expect modest gains.

Key Idea

Tuning picks the best *configuration of a model family* for your data. It cannot fix poor data, missing features, or a leaky evaluation. Because tuning is itself a form of fitting, it can also overfit, so most of the care in this reading goes into evaluating the search honestly.

2 Parameters and Hyper-parameters

2.1 The Distinction

A model's **parameters** are estimated from the training data by the learning algorithm: the intercept and slopes of linear regression, or the feature and cut-off chosen at each node of a decision tree. **Hyper-parameters** are fixed before training begins and control *how* the parameters are learned [8]. Scikit-learn supplies sensible defaults [10], but nothing guarantees they suit a given dataset.

2.2 Decision Trees and Random Forests

A single **decision tree** [4] has low bias and high variance: left at its defaults, it keeps splitting until every leaf is pure and memorizes the training set. Its tuning is mostly about *limiting growth*. A **random forest** [5] averages many trees, each grown on a bootstrap sample and allowed to consider only a random subset of features at each split. Averaging de-correlated trees reduces variance, so the individual trees can stay deep. For forests, `max_features` and the leaf-size settings usually matter most, while `n_estimators` mainly trades accuracy for computing time [12, 11]. Table 1 lists the key settings.

Hyper-parameter	**What it controls**	**Simpler model**
Decision tree (`DecisionTreeClassifier`)		
`max_depth`	Maximum number of levels; `None` grows until leaves are pure	Lower
`min_samples_leaf`	Minimum training records allowed in a leaf	Higher
`min_samples_split`	Minimum records a node needs before it may be split	Higher
`criterion`	Impurity measure: Gini or entropy	—
`ccp_alpha`	Strength of cost-complexity pruning	Higher
Random forest (`RandomForestClassifier`): all of the above, plus		
`n_estimators`	Number of trees; more trees reduce variance and cost time	—
`max_features`	Features considered at each split (`"sqrt"`, a fraction, or all)	Lower
`bootstrap`	Train each tree on a bootstrap sample or on all records	—

Table 1: Important hyper-parameters of decision trees and random forests in scikit-learn. The last column gives the direction that makes the model simpler (more regularized).

3 The Validation Protocol

3.1 Scoring Candidates with Cross-Validation

To compare settings we need a score that reflects performance on *new* data. Training accuracy cannot (a fully grown tree scores 100% on records it memorized), and neither can the test set, which stops being a fair judge once it is used to choose a model. The solution is **k-fold cross-validation on the training set only** [8], stratified for classification as in the Classification reading:

1. Split once into training and test sets, and set the test set aside.
2. For each candidate setting, run k-fold CV on the training set and record the mean and standard deviation of the score.

3. Choose the best setting and refit it on the entire training set.

4. Evaluate that single final model on the test set once, and report the result.

3.2 The Cost of a Search

A search over C candidate settings with k-fold CV fits $C \times k$ models, plus one final refit. Koehrsen's random search used 100 candidates and 3 folds, or 300 fits; a full grid over his six random-forest hyper-parameters would have held 4,320 candidates, or 12,960 fits [8]. Time one fit and multiply by $C \times k$ before starting; if the answer is hours, shrink the search or use the methods of Section 5.4.

4 Validation Curves

This reading uses the Breast Cancer Wisconsin data [14] from the Classification reading: 569 tumors, 30 features, and 37% malignant (the positive class). All results are *actual results* from a stratified 80/20 split (`random_state=42`; 455 training and 114 test tumors) with stratified 5-fold CV on the training set.

A **validation curve** varies one hyper-parameter, others at their defaults, and plots training and CV scores together, a controlled experiment in Koehrsen's sense [8]. Figure 1 shows two.

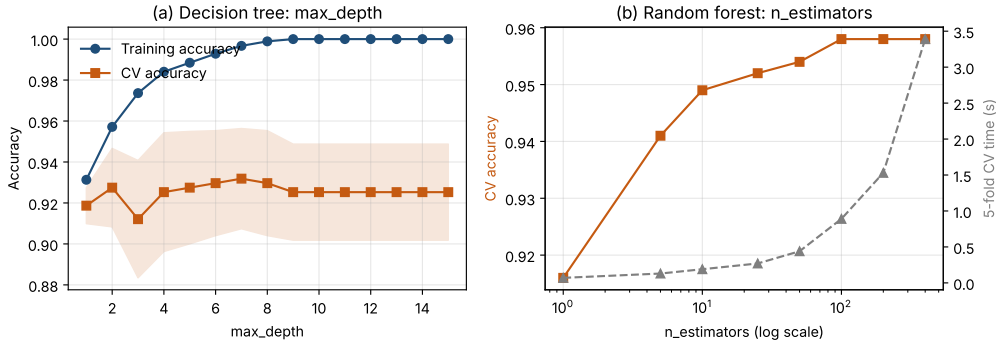


Figure 1: Validation curves on the Breast Cancer Wisconsin training data (actual results). (a) Decision tree: training accuracy reaches 1.0, while CV accuracy stays between 0.91 and 0.93 at every depth; the shaded band (±1 SD across folds) is wider than the differences between depths. (b) Random forest: CV accuracy rises from 0.916 with one tree to 0.958 at 100 trees and then stays flat, while the time for 5-fold CV keeps growing.

Panel (a) is the overfitting pattern of the Classification reading: extra depth buys training fit, not generalization (beyond depth 9 nothing changes, because the unrestricted fold trees never grow deeper). Panel (b) reproduces Koehrsen's observation that accuracy plateaus after a modest number of trees while cost keeps rising [8]. For `n_estimators`, the goal is "enough," not "optimal."

Common Pitfall

One knob at a time is not a search. A validation curve shows a single slice of the search space, and hyper-parameters interact: the best depth when `min_samples_leaf` is 1 is not the best depth when it is 10. Use validation curves to choose sensible *ranges*, then search the

combinations jointly.

5 Search Strategies

5.1 Grid Search

Grid search lists a few values for each hyper-parameter and evaluates every combination. It is exhaustive within the grid, easy to explain, and reproducible. Its weakness is combinatorial growth: the number of candidates is the *product* of the list lengths, so five hyper-parameters with four values each already give $4^5 = 1{,}024$ candidates.

5.2 Random Search

Random search draws a fixed number of combinations (`n_iter`) from a distribution for each hyper-parameter. Bergstra and Bengio showed that, for the same budget, it usually finds settings as good as or better than grid search, because in most problems only a few hyper-parameters matter much [2]. With nine trials, for example, a 3×3 grid tests only three distinct values of the important hyper-parameter, while nine random trials test nine.

On our data, with 20 candidates for the decision tree, a 5×4 grid reached a best CV accuracy of 0.934, while ten random searches of the same size averaged 0.939 (range 0.934 to 0.947): never worse, but within the fold SD of about 0.02.

5.3 Coarse-to-Fine Refinement

Koehrsen runs a wide random search first, then a small grid around its best values [8], a sensible default for classroom-sized problems. His second, more refined grid did slightly *worse* than the first: the sign of diminishing returns, which we meet again in Section 6.

5.4 Beyond Grid and Random Search

Bayesian optimization fits a surrogate model of the score to choose each next trial from earlier ones [13]; the Optuna library implements it [1]. **Successive halving** and **Hyperband** start many candidates on a small budget, such as a few trees or a data subsample, and keep only the most promising [9]; scikit-learn provides `HalvingGridSearchCV` and `HalvingRandomSearchCV`. They pay off when fits are expensive; for course-sized trees and forests, random search plus refinement is usually enough [3].

6 Python Implementation

The code below tunes a decision tree with grid search and a random forest with random search, following Koehrsen's workflow [8] in scikit-learn [10]. The full script, which reproduces every number and figure in this reading, accompanies it.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import (train_test_split, StratifiedKFold,
                                     GridSearchCV, RandomizedSearchCV)
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
```

```
from scipy.stats import randint

X, y = load_breast_cancer(return_X_y=True)
y = 1 - y                                   # malignant = positive class (1)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)

# Decision tree: grid search, 2 x 7 x 5 x 4 = 280 candidates (1,400 fits)
dt_grid = {"criterion": ["gini", "entropy"], "max_depth": [2, 3, 4, 5, 6, 8, None],
           "min_samples_leaf": [1, 2, 5, 10, 20], "ccp_alpha": [0, 0.005, 0.01, 0.02]}
gs_dt = GridSearchCV(DecisionTreeClassifier(random_state=0), dt_grid,
                     cv=cv, scoring="accuracy").fit(X_tr, y_tr)

# Random forest: random search, 40 candidates (200 fits)
rf_dist = {"n_estimators": randint(50, 501), "bootstrap": [True, False],
           "max_features": ["sqrt", "log2", 0.3, 0.5, None],
           "max_depth": [None, 4, 6, 8, 12, 20],
           "min_samples_split": randint(2, 21), "min_samples_leaf": randint(1, 11)}
rs_rf = RandomizedSearchCV(RandomForestClassifier(random_state=0), rf_dist,
                           n_iter=40, cv=cv, scoring="accuracy",
                           random_state=0).fit(X_tr, y_tr)

print(gs_dt.best_params_, gs_dt.best_score_)   # chosen on CV only
print(rs_rf.best_params_, rs_rf.best_score_)
print(gs_dt.score(X_te, y_te), rs_rf.score(X_te, y_te))  # test: once
```

Both searches refit the best setting on the full training set (`refit=True`). The forest search was followed by a 32-candidate refinement grid around its best values. Table 2 summarizes the outcome.

Model	CV accuracy	Test acc.	Test recall	Test AUC	Search time
Baseline (majority class)	—	0.632	0.000	0.500	—
Decision tree, defaults	0.925 ± 0.023	0.930	0.905	0.925	—
Decision tree, grid (280)	0.943 ± 0.023	0.939	0.833	0.968	5 s
Random forest, defaults	0.958 ± 0.008	0.965	0.905	0.994	—
Random forest, random (40)	**0.965** ± 0.027	**0.974**	**0.929**	**0.994**	96 s
Random forest, refined grid (32)	0.963	0.965	0.905	0.994	76 s

Table 2: Tuning results on Breast Cancer Wisconsin (actual results, scikit-learn 1.9). CV is mean 5-fold accuracy on the training set (± SD across folds); test metrics are for the single refitted model; recall is for the malignant class.

Real-World Case Study

Decision tree. The default tree (depth 8, 24 leaves) scores 1.000 on training data but 0.925 in CV. The grid chose entropy, `max_depth=4`, `min_samples_leaf=10`, and no pruning. The tuned tree has 9 leaves instead of 24, and gains 1.8 points of CV accuracy. On the test set, accuracy and AUC improved, but *recall fell* from 0.905 to 0.833: the tuned tree misses 7 of the 42 malignant tumors instead of 4. The search optimized exactly what it was asked to, and accuracy was the wrong request for a cancer diagnosis (the accuracy paradox of

the Classification reading). Pass `scoring="recall"`, `"f1"`, or `"roc_auc"` when that is what matters.

Random forest. Random search gained only 0.7 points of CV accuracy, less than the fold SD and close to Koehrsen's 0.4% [8]; the refinement grid then did slightly *worse*, as his second grid did. Forests perform well near their defaults [11].

Plateaus. 42 of the 280 tree candidates scored within 0.01 of the best; when settings tie, prefer the simplest.

7 Evaluating the Search Honestly

7.1 The Winner's Score Is Optimistic

The best CV score of a search is the *maximum* of many noisy estimates, so it is biased upward: some candidates win partly by luck on these particular folds [15]. The more candidates, the larger the bias. **Nested cross-validation** measures it. An outer CV loop holds out each fold in turn, the complete grid search runs on the remaining data, and the chosen model is scored on the held-out outer fold [15]:

```
outer = StratifiedKFold(n_splits=5, shuffle=True, random_state=1)
search = GridSearchCV(DecisionTreeClassifier(random_state=0), dt_grid, cv=cv)
nested = cross_val_score(search, X_tr, y_tr, cv=outer)  # 7,000 fits
```

For the decision tree, the nested estimate is **0.914 (±0.022)**, against 0.943 for the grid's best candidate: a three-point optimism from selecting the best of 280 settings. Nested CV scores the *tuning procedure*; it does not choose the final hyper-parameters.

7.2 Never Tune on the Test Set

Had we picked the tree setting with the best *test* accuracy, we would report 0.965 instead of the honest 0.939, level with the random forest. That number is a selection made with the answers in hand, not a performance estimate [15]. Looking at test results, adjusting the grid, and searching again leaks in the same way.

8 Common Pitfalls

- **Searching the wrong range.** If the best value sits at the edge of the grid, widen the range and search again.
- **Optimizing the wrong metric.** The search returns the best model for the `scoring` you set, as the tuned tree's lower recall showed.
- **Touching the test set during tuning.** Any choice informed by test results biases the final estimate.
- **Preprocessing outside the folds.** Scaling, imputation, and feature selection must be fitted inside each fold: put them in a `Pipeline` and tune names such as `"model__max_depth"`, as in the Feature Selection reading.

- **Tuning without a stopping rule.** Stop when another round gains less than the noise level or the model is good enough for the application. No setting beats the irreducible (Bayes) error of the problem [8].

9 Key Takeaways

1. Hyper-parameters are set before training; parameters are learned. Defaults are a starting point, not an answer.
2. Choose settings by cross-validation on the training set, and evaluate the final model on the test set once.
3. Decision trees need their growth limited (depth, leaf size, pruning). Random forests are strong near their defaults, and the number of trees only needs to be large enough.
4. Use validation curves to set ranges, random search to explore, and a small grid to refine. The best CV score of a search is optimistic. Nested CV estimates how well the tuning procedure generalizes.
5. Tuning gains are often small next to gains from better data and features; stop when improvements fall within the noise.

Wrap-Up: Connecting Back

Hyper-parameter optimization makes the complexity control of the Regression and Classification readings systematic, and the leakage rules of the Feature Selection reading decide where the search may look. Later models bring new hyper-parameters, but the protocol stays the same: search on training data, judge with cross-validation, test once.

Review Questions

1. Classify each as a parameter or a hyper-parameter of a random forest: the cut-off at a node of tree 17; `max_features`; the number of trees; the class predicted by a leaf.
2. A grid has 3 values of `n_estimators`, 4 of `max_depth`, 3 of `min_samples_leaf`, and 2 of `bootstrap`. With 5-fold CV, how many models are fitted, including the final refit?
3. The tuned decision tree had higher accuracy but lower recall than the default tree. What would you change in the search, and why?
4. Why is the nested-CV estimate (0.914) lower than the grid's best CV score (0.943)? Which would you report?

Suggested Hands-on Exercise

Tuning trees and forests (individual or pairs). Use the Pima Indians Diabetes data from the Classification reading, or a binary dataset from your project domain. Then:

1. **Split** with stratification, set the test set aside, and **plot validation curves** for decision-tree `max_depth` and random-forest `max_features`.

2. **Tune** a decision tree with `GridSearchCV` and a random forest with `RandomizedSearchCV` (`n_iter=30`), both with `scoring="roc_auc"`.

3. **Compare** defaults and tuned models in a table like Table 2 (CV and test ROC AUC, test recall). **Estimate** one search with nested CV, compare it with the search's best CV score, and **judge** in a short paragraph whether tuning was worth its computing time.

References

[1] Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2623–2631. https://doi.org/10.1145/3292500.3330701

[2] Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13, 281–305. https://jmlr.org/papers/v13/bergstra12a.html

[3] Bischl, B., Binder, M., Lang, M., Pielok, T., Richter, J., Coors, S., et al. (2023). Hyperparameter optimization: Foundations, algorithms, best practices, and open challenges. *WIREs Data Mining and Knowledge Discovery*, 13(2), e1484. https://doi.org/10.1002/widm.1484

[4] Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth.

[5] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324

[6] Han, J., Pei, J., & Tong, H. (2023). *Data Mining: Concepts and Techniques* (4th ed.). Morgan Kaufmann.

[7] Hasan, K. (2026). *Knowledge Discovery and Data Mining*. Grand Valley State University. Available at: https://github.com/mdkamrulhasan/data_mining_kdd

[8] Koehrsen, W. (2018, January 9). Hyperparameter tuning the random forest in Python. *Towards Data Science*. https://towardsdatascience.com/hyperparameter-tuning-the-random-forest-in-python-using-scikit-learn-28d2aa77dd74/

[9] Li, L., Jamieson, K., DeSalvo, G., Rostamizadeh, A., & Talwalkar, A. (2018). Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185), 1–52. https://jmlr.org/papers/v18/16-558.html

[10] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[11] Probst, P., Boulesteix, A.-L., & Bischl, B. (2019). Tunability: Importance of hyperparameters of machine learning algorithms. *Journal of Machine Learning Research*, 20(53), 1–32. https://jmlr.org/papers/v20/18-444.html

[12] Probst, P., Wright, M. N., & Boulesteix, A.-L. (2019). Hyperparameters and tuning strategies for random forest. *WIREs Data Mining and Knowledge Discovery*, 9(3), e1301. https://doi.org/10.1002/widm.1301

[13] Snoek, J., Larochelle, H., & Adams, R. P. (2012). Practical Bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems 25*, 2951–2959.

[14] Street, W. N., Wolberg, W. H., & Mangasarian, O. L. (1993). Nuclear feature extraction for breast tumor diagnosis. In *Biomedical Image Processing and Biomedical Visualization*, Proc. SPIE 1905, 861–870. https://doi.org/10.1117/12.148698

[15] Varma, S., & Simon, R. (2006). Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7, 91. https://doi.org/10.1186/1471-2105-7-91